

Branch and Bound for flowshop

```
In [ ]: import matplotlib.pyplot as plt
```

References:

- This paper : <https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=&cad=rja&uact=8&ved=2ahUKEwil7rOSz9GEAxWVVKQEHRNtCi8QFnoECBoQAAQ&url=https%3A%2F0BtHd&opi=89978449>
- This youtube video : <https://www.youtube.com/watch?v=Q58zRyoa4IE&t=2838s>

Implementing the flowshop with branch and bound class

```
In [ ]: import copy

class FlowShopBB:
    def __init__(self, jobs_list:list):
        self.jobs_list = jobs_list
        self.upper_bound = self.comp_ub(jobs_list)
        self.seq_star = None
        self.debug = False
        self.array_make_span=[]

    #compute initial upper bound , to improve Later ...
    def comp_ub(self, jobs_list:list):
        upper_bound = 0
        for j in range(len(jobs_list[0])):
            for i in range(len(jobs_list)):
                upper_bound += jobs_list[i][j]
        return upper_bound

    #compute the lower bound of the makespan for a partial sequence given its gantt cumule
    def lower_bound(self, jobs_sequence:list, jobs_seq_cumule:list, previous_cumul:list=None):
        nb_machines = len(self.jobs_list[0])
        lb = 0

        if(len(self.jobs_list) - len(jobs_sequence) != 0):
            remaining_time = [0] * (len(self.jobs_list) - len(jobs_sequence)) # the number of remaining jobs
        else:
            remaining_time = [0] # this needs to be a List for the min() function

        for machine_index in range( nb_machines ):
            cumulative_time_to_machine = jobs_seq_cumule[machine_index]
            remaining_machine_time = 0

            counter = 0
            for i in range(len(self.jobs_list)):
                if i not in jobs_sequence:
                    remaining_machine_time += self.jobs_list[i][machine_index]
                    remaining_time[counter] = 0
                    for j in range(machine_index + 1, nb_machines):
                        remaining_time[counter] += self.jobs_list[i][j]
                    counter += 1

            lb_machine = cumulative_time_to_machine + remaining_machine_time + min(remaining_time)
            lb = max(lb, lb_machine)

        return lb

    # utility function that returns the gantt cumule based on a job execution times and a previous gantt cumule
    def cumulate(self, job:list, previous_cumul=None):
        res = [0] * len(job)

        if(previous_cumul!=None):
            res[0] = job[0]
            for i in range(1, len(job)):
                res[i] = res[i-1] + job[i]
        else:
            res[0] = previous_cumul[0] + job[0]
            for i in range(1, len(job)):
                res[i] = max(res[i-1], previous_cumul[i]) + job[i]
```

```

    return res

# utility function that computes the gantt cumule given only a job sequence (not used in the algorithm due to inefficiency
# dynamic programming with cumulate is used instead ...)
def cumulate_seq(self, seq:list):
    cumulated = None
    for i in seq:
        cumulated = self.cumulate(self.jobs_list[i], cumulated)

    return cumulated

#branch and bound
def bb(self, job_seq:list, previous_cumul:list):
    if len(job_seq) == len(self.jobs_list): #it's a complete sequence
        if previous_cumul[-1] < self.upper_bound:
            self.seq_star = copy.deepcopy(job_seq)
            self.upper_bound = previous_cumul[-1]
            self.array_make_span.append(previous_cumul[-1])
            if(self.debug == True):
                print("new seq_star :", self.seq_star)
                print("new makespan :", previous_cumul[-1])
                print("=====")
        else:
            augmented_job_seqs = []
            for i in range(len(self.jobs_list)):
                if i not in job_seq:
                    augmented_partial_set=list (copy.deepcopy(job_seq))
                    augmented_partial_set.append(i)
                    cumu = self.cumulate(self.jobs_list[i], previous_cumul)
                    augmented_job_seqs.append( ( self.lower_bound(augmented_partial_set, cumu), augmented_partial_set, cumu ) )

            ordered_augmented_job_seqs = sorted(augmented_job_seqs)

            for lb_seq_cumu_tuple in ordered_augmented_job_seqs:
                lb = lb_seq_cumu_tuple[0]
                seq = lb_seq_cumu_tuple[1]
                cumu = lb_seq_cumu_tuple[2]
                # if the lower bound of the partial sequence is less than the best MakeSpan(self.upper_bound)
                # that we found until now, we develop the branch, otherwise we prune
                if lb < self.upper_bound:
                    self.bb(seq, cumu)

#Launching the optimization : this function initializes the singleton partial sequences, and calls branch and bound (bb)
def optim(self, debug=False):
    self.debug = debug
    if (len(self.jobs_list)==1):
        self.seq_star = [0]

    else:
        initial_job_sequences = []
        for i in range(len(self.jobs_list)):
            cumu = self.cumulate(self.jobs_list[i], previous_cumul=None)
            initial_job_sequences.append( ( self.lower_bound([i], cumu), [i], cumu ) )

        ordered_initial_job_sequences = sorted(initial_job_sequences)

        for lb_seq_cumul_tuple in ordered_initial_job_sequences:
            lb = lb_seq_cumul_tuple[0]
            seq = lb_seq_cumul_tuple[1]
            cumu = lb_seq_cumul_tuple[2]
            # if the lower bound of the partial sequence is less than the best MakeSpan(self.upper_bound)
            # that we found until now, we develop the branch, otherwise we prune
            if lb < self.upper_bound:
                self.bb(seq, cumu)

```

Definition of some utility functions and data structures

Getting all the instances of Taillard

```

In [ ]: import time

# class for holding the data for each instance of the benchmark
class Inst:

```

```

def __init__(self, jobs: int, machines: int, seed: int, ub: int, lb: int, matrix: list[list[int]]):
    self.jobs = jobs
    self.machines = machines
    self.seed = seed
    self.ub = ub
    self.lb = lb
    self.matrix = matrix

# Loading the instances from the benchmark file
with open("ben.txt", "r") as f:
    lines = f.readlines()

instances: list[Inst] = []
i = 0
while i < len(lines):
    params = [int(e) for e in lines[i].split()]
    jobs = params[0]
    machines = params[1]
    seed = params[2]
    ub = params[3]
    lb = params[4]

    matrix = [[int(e) for e in line.strip().split()] for line in lines[i+3:i+3+machines]]
    instances.append(Inst(jobs, machines, seed, ub, lb, matrix))
    i+=3+machines

```

Function to get the jobs list of one instance

```

In [ ]: def get_instance(k):
        jobs_list = []
        for i in range(len(instances[k].matrix[0])):
            jobs_list.append([])
            for j in range(len(instances[k].matrix)):
                jobs_list[-1].append(instances[k].matrix[j][i])
        return jobs_list

```

Function for plotting execution times and evolution of MakeSpan

```

In [ ]: def plotting_time_for_each_number_jobs(x_values,y_values):
        plt.plot(x_values, y_values, marker='o')

        # Adding Labels and title
        plt.xlabel('number of jobs')
        plt.ylabel('Time (s)')
        plt.title('the execution time for each number of jobs ')

        # Display the graph
        plt.show()

```

```

In [ ]: def plotting_make_span_at_each_step(y_values,i):
        plt.plot(y_values, marker='o')

        # Adding Labels and title
        plt.xlabel('step')
        plt.ylabel('Make Span')
        plt.title(f'the make span when number of jobs equal to {i}')

        # Display the graph
        plt.show()

```

Function for executing branch and bound for an instance

```

In [ ]: def exec_instance(k):
        jobs_list = get_instance(k)

        fs = FlowShopBB(jobs_list)
        print("Starting optim of the 20 jobs ...")
        start = time.time()
        fs.optim(debug=True)
        print("Execution time: ", time.time() - start)
        print("seq_star:", fs.seq_star)
        print("MakeSpan:",fs.cumulate_seq(fs.seq_star)[-1])

```

Testing using a small test case

```
In [ ]: #the jobs List structure, each row "i" is the time of executions of job Ji for every machine Mj
jobs_list = [[5,6,4],
             [8,7,8],
             [7,2,7],
             [3,5,9]]

fs = FlowShopBB(jobs_list)
print("Starting optim of the 4 jobs ...")
start = time.time()

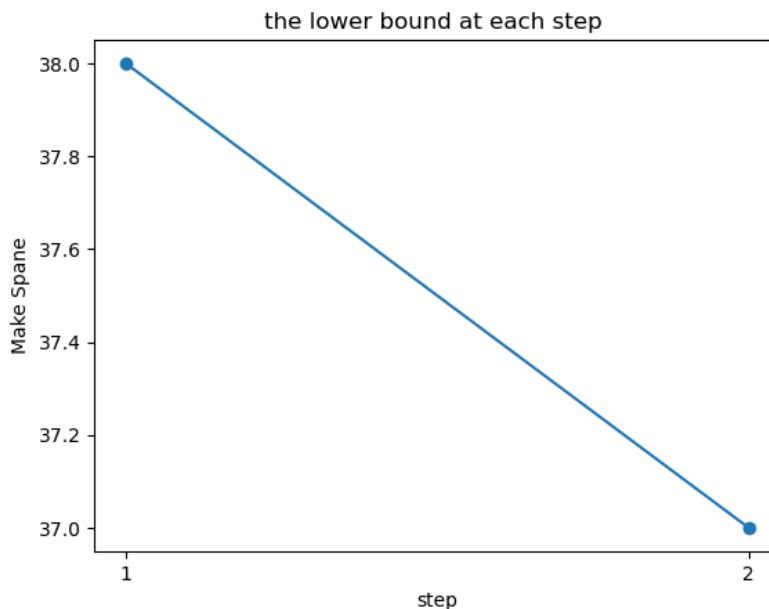
# We launch the optimization by calling the method .optim()
# setting debug = True let's us print every time the algorithm finds a new best sequence (seq_star) during exploration
# We print at the end the execution time in seconds, the optimal sequence (seq_star) and its Makespan
fs.optim(debug=True)
print("Execution time: ", time.time() - start)
print("seq_star:", fs.seq_star)
print("MakeSpan:", fs.upper_bound)

Starting optim of the 4 jobs ...
new seq_star : [3, 0, 1, 2]
new makespan : 38
=====
new seq_star : [3, 2, 1, 0]
new makespan : 37
=====
Execution time:  0.0
seq_star: [3, 2, 1, 0]
MakeSpan: 37
```

```
In [ ]: plt.plot([1,2], fs.array_make_span, marker='o')

# Adding labels and title
plt.xlabel('step')
plt.ylabel('Make Spane')
plt.title('the lower bound at each step ')

plt.xticks(range(min([1,2]), max([1,2])+1))
# Display the graph
plt.show()
```

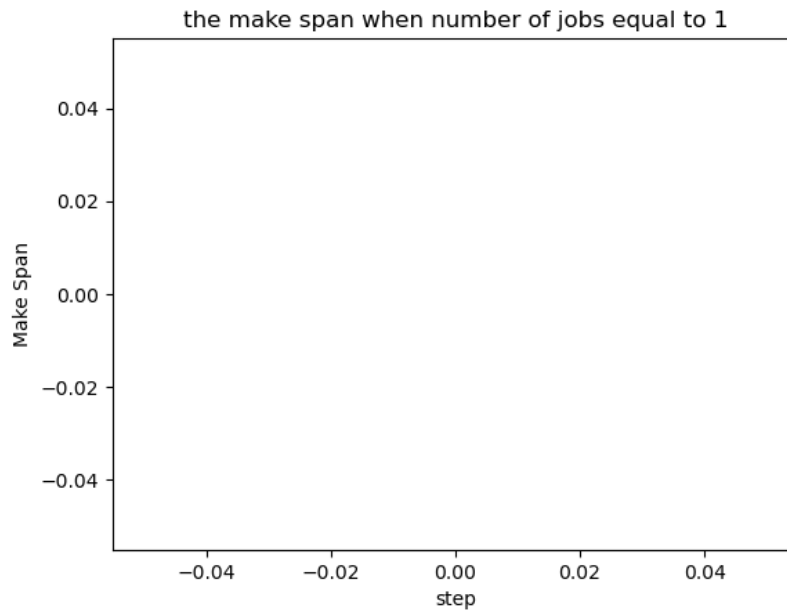


Launching the branch and bound for the first Taillard instance for different number of jobs (from 1 to 12)

Executing and plotting the evolution of MakeSpan during branch and bound

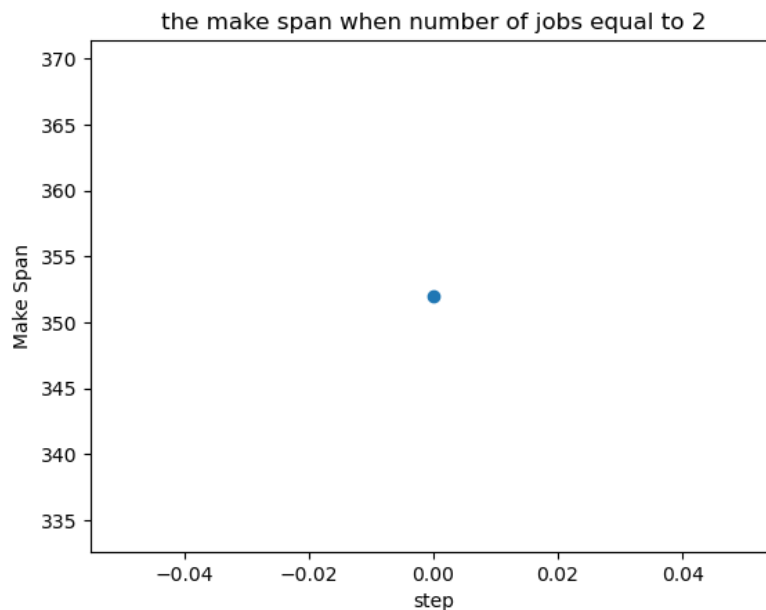
```
In [ ]: # here we will launch the execution of branch and bound for gradually increasing number of jobs in the first instance
# The execution gets longer as the number of jobs increases, thus we stop it manually once it starts to take too long
jobs_list=get_instance(0)
times=[]
x_values=[]
for i in range(len(jobs_list)):
    local_jobs_list = jobs_list[:i+1]
    print("=====")
    print(f"Starting optim of the first {i+1} jobs ...")
    fs = FlowShopBB(local_jobs_list)
    start = time.time()
    fs.optim(False)
    end = time.time() - start
    times.append(end)
    plotting_make_span_at_each_step(fs.array_make_span,i+1)
    print("Execution time: ", end)
    print("seq_star:", fs.seq_star)
    print("MakeSpan:", fs.upper_bound)
    x_values.append(i+1)
    if(i==11) : break
```

=====
Starting optim of the first 1 jobs ...



Execution time: 0.0
seq_star: [0]
MakeSpan: 273

=====
Starting optim of the first 2 jobs ...



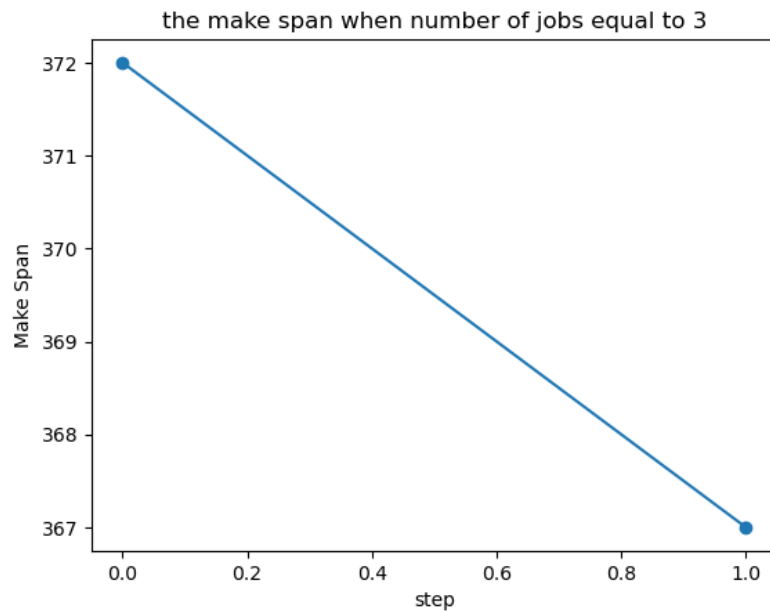
Execution time: 0.0

seq_star: [0, 1]

MakeSpan: 352

=====

Starting optim of the first 3 jobs ...



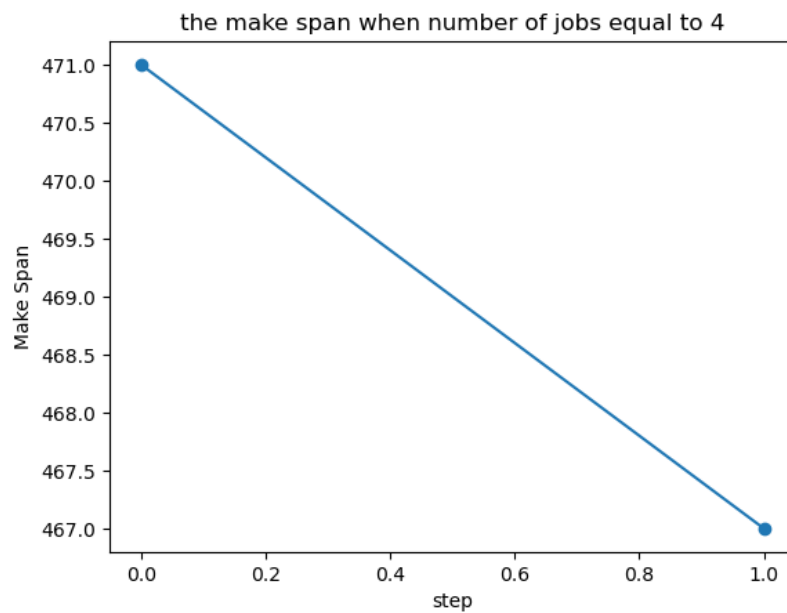
Execution time: 0.0

seq_star: [2, 0, 1]

MakeSpan: 367

=====

Starting optim of the first 4 jobs ...



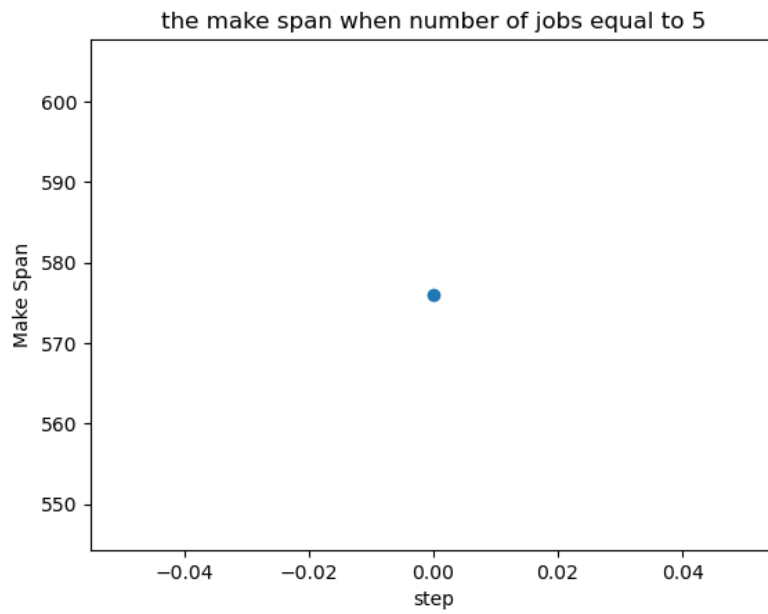
Execution time: 0.002520322799682617

seq_star: [0, 2, 3, 1]

MakeSpan: 467

=====

Starting optim of the first 5 jobs ...



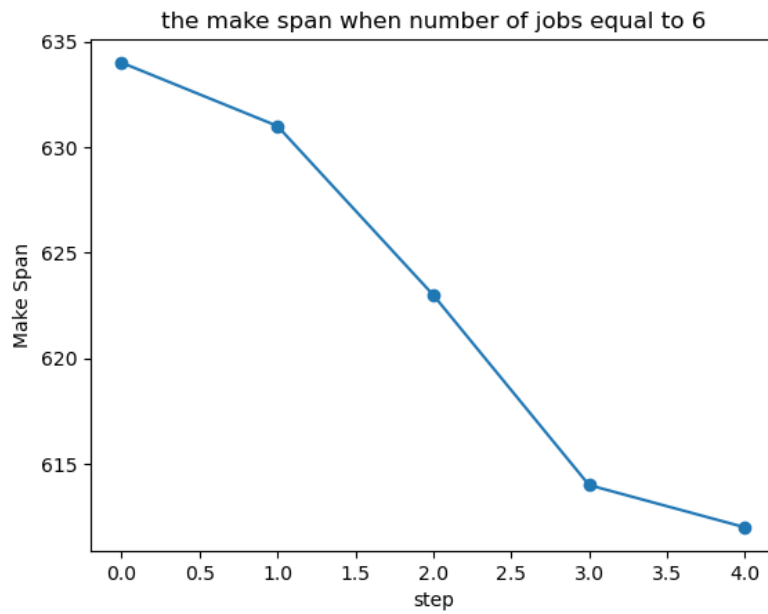
Execution time: 0.01150965690612793

seq_star: [0, 2, 3, 1, 4]

MakeSpan: 576

=====

Starting optim of the first 6 jobs ...



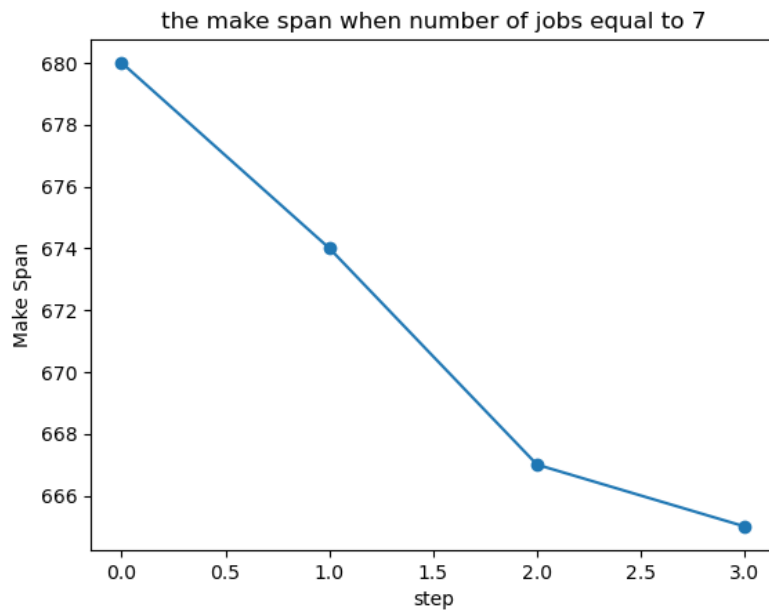
Execution time: 0.016600847244262695

seq_star: [5, 3, 1, 0, 2, 4]

MakeSpan: 612

=====

Starting optim of the first 7 jobs ...



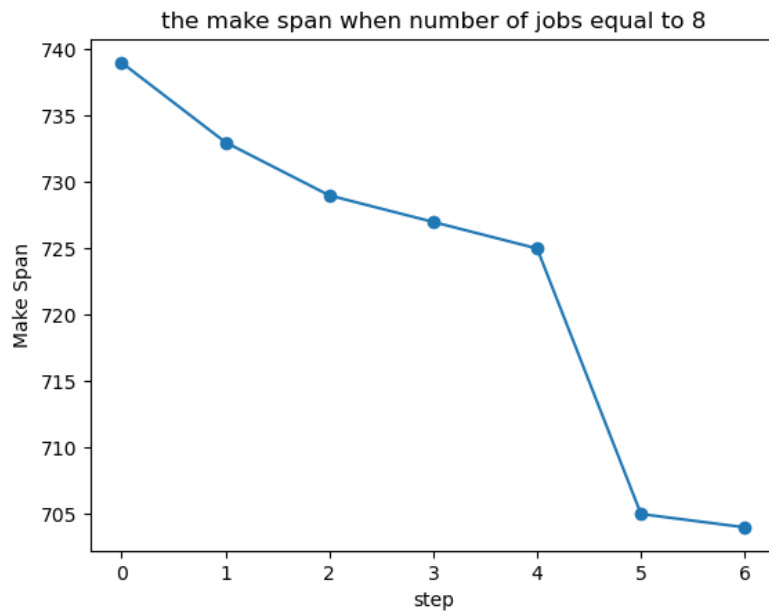
Execution time: 0.054840087890625

seq_star: [5, 3, 1, 0, 2, 4, 6]

MakeSpan: 665

=====

Starting optim of the first 8 jobs ...



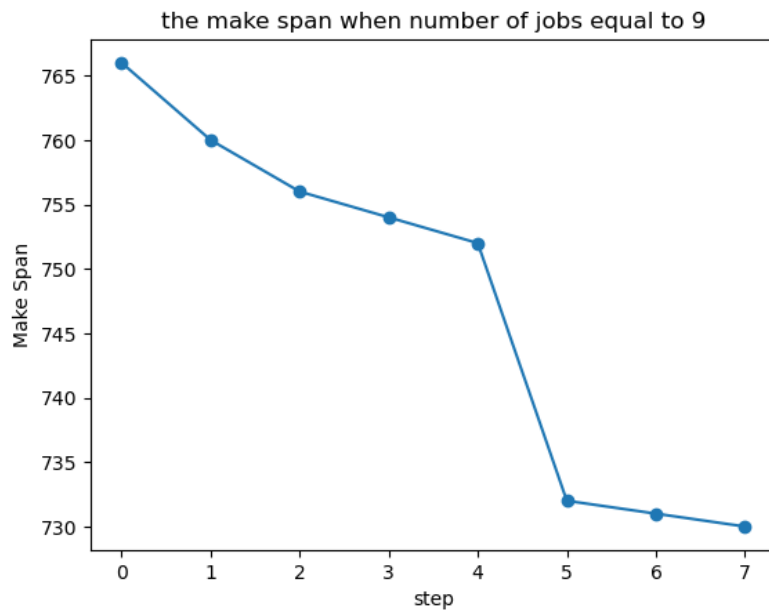
Execution time: 0.1659867763519287

seq_star: [2, 5, 0, 3, 1, 7, 4, 6]

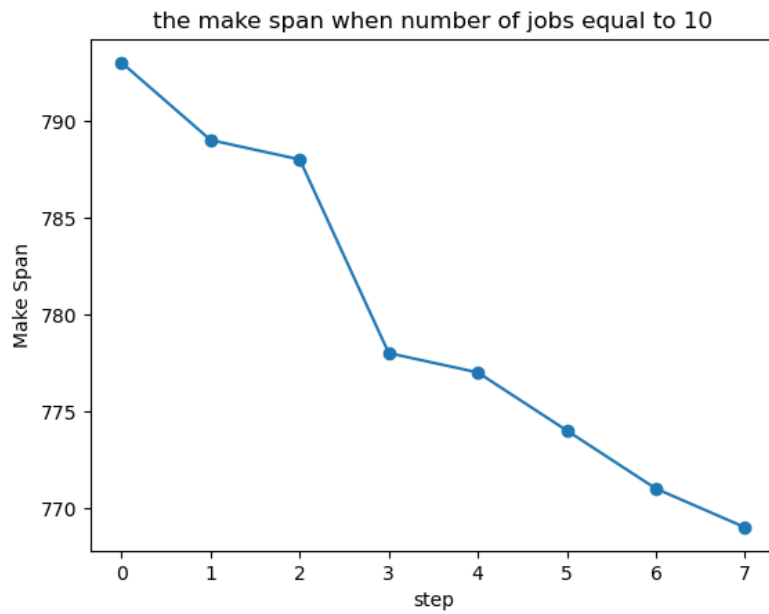
MakeSpan: 704

=====

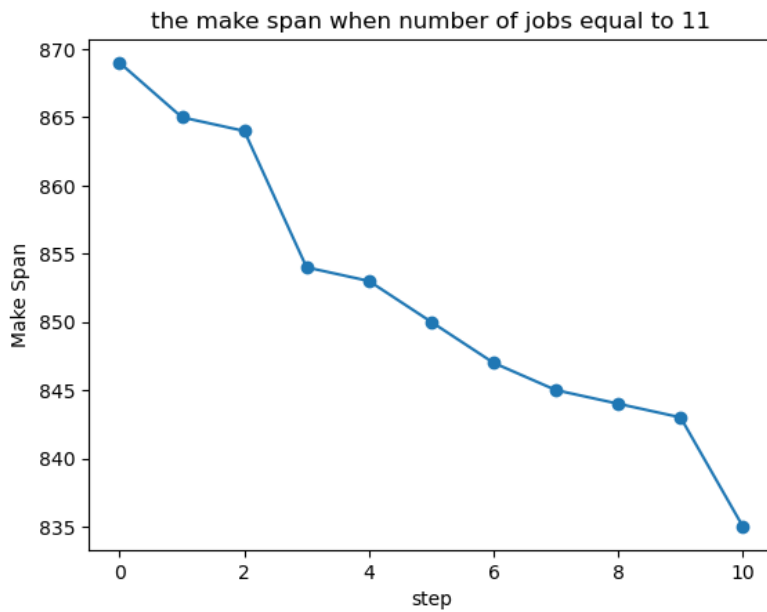
Starting optim of the first 9 jobs ...



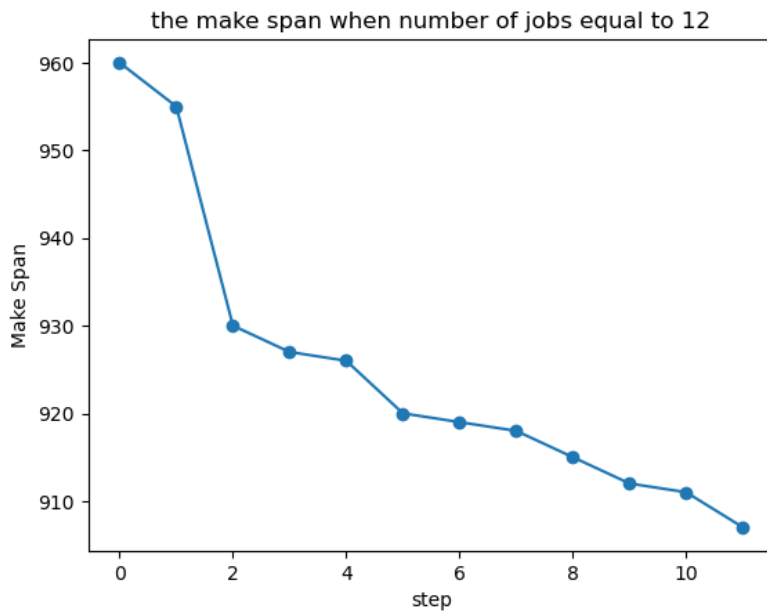
Execution time: 0.3657364845275879
 seq_star: [7, 8, 5, 3, 1, 0, 2, 4, 6]
 MakeSpan: 730
 =====
 Starting optim of the first 10 jobs ...



Execution time: 0.8834280967712402
 seq_star: [2, 7, 0, 8, 5, 4, 3, 1, 9, 6]
 MakeSpan: 769
 =====
 Starting optim of the first 11 jobs ...



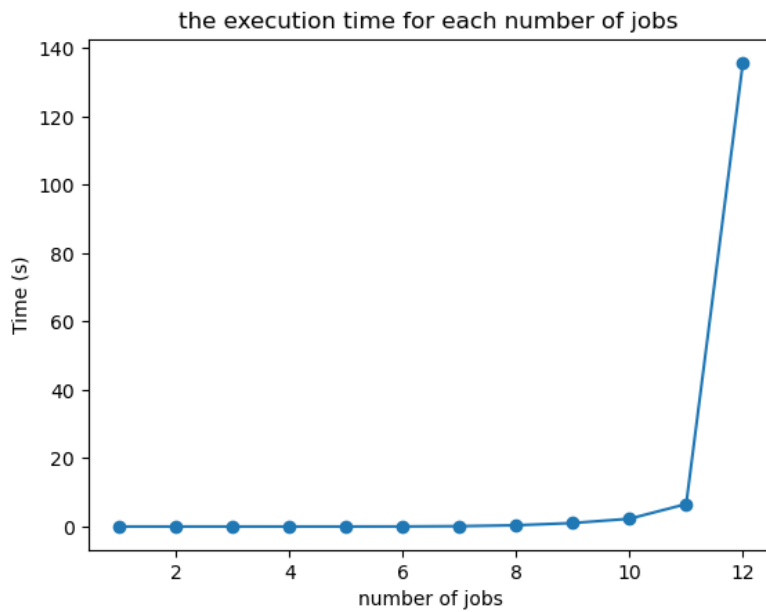
Execution time: 2.556377410888672
 seq_star: [8, 7, 2, 5, 3, 1, 0, 4, 6, 10, 9]
 MakeSpan: 835
 =====
 Starting optim of the first 12 jobs ...



Execution time: 52.327046394348145
 seq_star: [8, 7, 2, 5, 6, 10, 0, 4, 3, 1, 9, 11]
 MakeSpan: 907

Plotting the evolution of the execution time for each number of jobs

```
In [ ]: plotting_time_for_each_number_jobs(x_values,times)
```



Launching the execution for every instance of Taillard (we stop manually when it takes too long)

In []: `exec_instance(0)`

```
Starting optim of the 20 jobs ...
new seq_star : [7, 0, 8, 3, 10, 1, 12, 15, 5, 14, 16, 18, 13, 4, 6, 19, 11, 2, 17, 9]
new makespan : 1370
=====
new seq_star : [7, 0, 8, 3, 10, 1, 12, 15, 5, 14, 16, 18, 13, 4, 6, 19, 2, 9, 17, 11]
new makespan : 1358
=====
new seq_star : [7, 0, 8, 3, 10, 1, 12, 15, 5, 14, 16, 18, 13, 4, 6, 19, 2, 17, 11, 9]
new makespan : 1339
=====
new seq_star : [7, 0, 8, 3, 10, 1, 12, 15, 5, 14, 16, 18, 13, 4, 6, 11, 9, 2, 17, 19]
new makespan : 1324
=====
new seq_star : [7, 0, 8, 3, 10, 1, 12, 15, 5, 14, 16, 18, 13, 4, 6, 11, 2, 17, 9, 19]
new makespan : 1322
=====
new seq_star : [7, 0, 8, 3, 10, 1, 12, 15, 5, 14, 16, 18, 13, 4, 6, 2, 17, 11, 9, 19]
new makespan : 1306
=====
new seq_star : [7, 0, 8, 3, 10, 1, 12, 15, 5, 14, 16, 18, 13, 4, 6, 17, 2, 11, 9, 19]
new makespan : 1297
=====
```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[14], line 1
----> 1 exec_instance(0)

Cell In[8], line 7, in exec_instance(k)
      5 print("Starting optim of the 20 jobs ...")
      6 start = time.time()
----> 7 fs.optim(debug=True)
      8 print("Execution time: ", time.time() - start)
      9 print("Seq_star:", fs.seq_star)

Cell In[3], line 124, in FlowShopBB.optim(self, debug)
     121 cumu = lb_seq_cumul_tuple[2]
     122 if lb < self.upper_bound:
     123     # cumu = self.cumulate(self.jobs_list[seq[-1]], previous_cumul=None) # here previous_cumul is none
--> 124     self.bb(seq, cumu)

Cell In[3], line 99, in FlowShopBB.bb(self, job_seq, previous_cumul)
      97 cumu = lb_seq_cumul_tuple[2]
      98 if lb < self.upper_bound:
----> 99     self.bb(seq, cumu)

Cell In[3], line 99, in FlowShopBB.bb(self, job_seq, previous_cumul)
      97 cumu = lb_seq_cumul_tuple[2]
      98 if lb < self.upper_bound:
----> 99     self.bb(seq, cumu)

[... skipping similar frames: FlowShopBB.bb at line 99 (13 times)]

Cell In[3], line 99, in FlowShopBB.bb(self, job_seq, previous_cumul)
      97 cumu = lb_seq_cumul_tuple[2]
      98 if lb < self.upper_bound:
----> 99     self.bb(seq, cumu)

Cell In[3], line 88, in FlowShopBB.bb(self, job_seq, previous_cumul)
      86 augmented_partial_set:list = (copy.deepcopy(job_seq))
      87 cumu = augmented_partial_set.append(i)
----> 88     cumu = self.cumulate(self.jobs_list[i], previous_cumul)
      89 augmented_job_seqs.append( ( self.lower_bound(augmented_partial_set, cumu), augmented_partial_set, cumu ) )
      90 ordered_augmented_job_seqs = sorted(augmented_job_seqs)

Cell In[3], line 59, in FlowShopBB.cumulate(self, job, previous_cumul)
      57 res[0] = previous_cumul[0] + job[0]
      58 for i in range(1, len(job)):
----> 59     res[i] = max(res[i-1], previous_cumul[i]) + job[i]
      61 return res

KeyboardInterrupt:

```

```
In [ ]: exec_instance(1)
```

```

Starting optim of the 20 jobs ...
new seq_star : [13, 5, 17, 18, 9, 6, 2, 15, 14, 3, 7, 16, 19, 0, 10, 12, 4, 11, 8, 1]
new makespan : 1457
=====
new seq_star : [13, 5, 17, 18, 9, 6, 2, 15, 14, 3, 7, 16, 19, 0, 10, 12, 4, 8, 11, 1]
new makespan : 1453
=====
new seq_star : [13, 5, 17, 18, 9, 6, 2, 15, 14, 3, 7, 16, 19, 0, 10, 12, 8, 4, 11, 1]
new makespan : 1451
=====
new seq_star : [13, 5, 17, 18, 9, 6, 2, 15, 14, 3, 7, 16, 19, 0, 10, 12, 8, 11, 4, 1]
new makespan : 1434
=====
new seq_star : [13, 5, 17, 18, 9, 6, 2, 15, 14, 3, 16, 4, 7, 19, 10, 11, 12, 8, 0, 1]
new makespan : 1430
=====
new seq_star : [13, 5, 17, 18, 9, 6, 2, 15, 14, 3, 16, 4, 7, 19, 11, 12, 8, 0, 10, 1]
new makespan : 1422
=====
new seq_star : [13, 5, 17, 18, 9, 6, 2, 15, 14, 3, 16, 8, 0, 7, 19, 10, 12, 11, 4, 1]
new makespan : 1420
=====
new seq_star : [13, 5, 17, 18, 9, 6, 2, 15, 14, 3, 16, 8, 0, 10, 12, 11, 7, 19, 4, 1]
new makespan : 1417
=====
new seq_star : [13, 5, 17, 9, 6, 2, 7, 18, 3, 16, 19, 10, 11, 15, 14, 0, 12, 8, 4, 1]
new makespan : 1409
=====
new seq_star : [13, 5, 17, 9, 6, 2, 7, 18, 3, 16, 19, 11, 10, 15, 14, 0, 12, 8, 4, 1]
new makespan : 1408
=====
new seq_star : [13, 5, 18, 9, 6, 2, 15, 11, 10, 16, 17, 14, 3, 0, 7, 8, 12, 19, 4, 1]
new makespan : 1391
=====
new seq_star : [13, 5, 9, 16, 6, 2, 15, 3, 18, 11, 10, 17, 14, 0, 7, 19, 12, 8, 4, 1]
new makespan : 1390
=====
new seq_star : [5, 13, 18, 9, 6, 2, 15, 11, 10, 16, 17, 14, 3, 0, 7, 8, 12, 4, 19, 1]
new makespan : 1388
=====
new seq_star : [5, 13, 18, 9, 6, 2, 15, 11, 10, 16, 17, 14, 3, 0, 7, 8, 12, 19, 4, 1]
new makespan : 1370
=====
new seq_star : [5, 13, 16, 18, 9, 6, 8, 2, 3, 7, 17, 14, 10, 11, 12, 15, 19, 0, 4, 1]
new makespan : 1365
=====
new seq_star : [5, 18, 13, 9, 6, 2, 15, 19, 10, 16, 4, 17, 14, 11, 0, 7, 8, 12, 3, 1]
new makespan : 1364
=====
new seq_star : [5, 18, 9, 6, 2, 13, 15, 11, 10, 16, 17, 19, 4, 0, 12, 8, 14, 3, 7, 1]
new makespan : 1360
=====
new seq_star : [5, 9, 16, 6, 2, 13, 15, 3, 18, 14, 8, 7, 17, 19, 10, 0, 12, 11, 4, 1]
new makespan : 1359
=====
Execution time: 0.4557797908782959
seq_star: [5, 9, 16, 6, 2, 13, 15, 3, 18, 14, 8, 7, 17, 19, 10, 0, 12, 11, 4, 1]
MakeSpan: 1359

```

```
In [ ]: exec_instance(2)
```

```

Starting optim of the 20 jobs ...
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 5, 7, 17, 8, 16, 4, 11, 1]
new makespan : 1301
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 5, 7, 17, 8, 16, 11, 4, 1]
new makespan : 1277
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 5, 7, 17, 16, 4, 11, 1, 8]
new makespan : 1262
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 5, 7, 17, 16, 11, 4, 1, 8]
new makespan : 1240
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 5, 17, 7, 4, 11, 8, 16, 1]
new makespan : 1236
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 5, 17, 16, 4, 11, 1, 7, 8]
new makespan : 1233
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 5, 17, 16, 11, 7, 4, 1, 8]
new makespan : 1223
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 5, 17, 16, 11, 4, 1, 8, 7]
new makespan : 1209
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 5, 17, 11, 4, 7, 16, 1, 8]
new makespan : 1207
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 7, 17, 16, 11, 4, 1, 8, 5]
new makespan : 1200
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 16, 17, 11, 5, 4, 1, 8, 7]
new makespan : 1197
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 16, 17, 11, 7, 4, 5, 8, 1]
new makespan : 1188
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 16, 17, 11, 4, 1, 5, 8, 7]
new makespan : 1187
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 17, 16, 11, 7, 4, 5, 8, 1]
new makespan : 1177
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 6, 17, 11, 4, 7, 16, 5, 8, 1]
new makespan : 1172
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 17, 4, 6, 11, 7, 16, 5, 8, 1]
new makespan : 1165
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 0, 19, 17, 6, 11, 4, 7, 16, 5, 8, 1]
new makespan : 1158
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 10, 17, 19, 11, 0, 6, 4, 7, 16, 5, 8, 1]
new makespan : 1157
=====
new seq_star : [2, 3, 9, 14, 18, 13, 12, 15, 17, 19, 6, 0, 4, 11, 5, 7, 16, 10, 8, 1]
new makespan : 1156
=====
new seq_star : [2, 3, 9, 14, 15, 10, 0, 18, 13, 12, 19, 17, 4, 6, 11, 7, 16, 5, 8, 1]
new makespan : 1153
=====
new seq_star : [2, 3, 9, 14, 15, 10, 0, 18, 13, 12, 19, 17, 6, 11, 4, 7, 16, 5, 8, 1]
new makespan : 1151
=====
new seq_star : [2, 3, 9, 14, 15, 10, 0, 18, 13, 12, 19, 17, 11, 6, 4, 7, 16, 5, 8, 1]
new makespan : 1149
=====
new seq_star : [2, 3, 9, 14, 15, 10, 0, 19, 6, 13, 12, 18, 17, 11, 4, 7, 16, 5, 8, 1]
new makespan : 1147
=====
new seq_star : [2, 3, 9, 14, 15, 10, 0, 19, 17, 6, 12, 13, 18, 11, 4, 7, 16, 5, 8, 1]
new makespan : 1144
=====
new seq_star : [2, 3, 9, 14, 15, 13, 12, 19, 17, 6, 0, 18, 4, 11, 5, 7, 16, 10, 8, 1]
new makespan : 1143
=====
new seq_star : [2, 3, 9, 15, 10, 0, 19, 6, 7, 18, 17, 4, 13, 12, 16, 11, 5, 8, 1, 14]
new makespan : 1141
=====
new seq_star : [2, 3, 9, 15, 10, 0, 19, 6, 7, 18, 17, 11, 12, 13, 16, 4, 5, 8, 1, 14]
new makespan : 1133

```

```

=====
new seq_star : [2, 3, 9, 15, 10, 0, 19, 6, 18, 13, 16, 17, 11, 12, 4, 5, 8, 7, 1, 14]
new makespan : 1130
=====
new seq_star : [2, 3, 9, 15, 10, 0, 19, 17, 6, 7, 18, 11, 12, 13, 16, 4, 5, 8, 1, 14]
new makespan : 1129
=====
new seq_star : [2, 3, 9, 15, 10, 12, 19, 13, 18, 17, 11, 0, 6, 16, 4, 5, 8, 7, 1, 14]
new makespan : 1128
=====
new seq_star : [2, 3, 9, 15, 17, 10, 0, 18, 6, 7, 19, 11, 12, 13, 16, 4, 5, 8, 1, 14]
new makespan : 1125
=====
new seq_star : [2, 3, 9, 15, 17, 10, 0, 18, 6, 19, 13, 16, 11, 12, 4, 5, 8, 7, 1, 14]
new makespan : 1121
=====
new seq_star : [2, 3, 9, 15, 17, 13, 12, 18, 19, 4, 0, 6, 11, 10, 16, 5, 8, 7, 1, 14]
new makespan : 1120
=====
new seq_star : [2, 3, 12, 15, 10, 0, 19, 17, 11, 9, 4, 13, 18, 6, 7, 16, 5, 8, 1, 14]
new makespan : 1117
=====
new seq_star : [2, 3, 14, 13, 12, 15, 10, 0, 19, 17, 6, 5, 18, 4, 9, 11, 8, 7, 16, 1]
new makespan : 1114
=====
new seq_star : [2, 3, 14, 18, 13, 9, 15, 10, 0, 19, 17, 4, 6, 12, 11, 8, 7, 16, 5, 1]
new makespan : 1111
=====
new seq_star : [2, 3, 18, 0, 15, 10, 9, 19, 17, 6, 11, 12, 13, 16, 4, 5, 8, 7, 14, 1]
new makespan : 1110
=====
new seq_star : [2, 3, 18, 0, 15, 10, 9, 19, 17, 11, 12, 13, 6, 16, 4, 5, 8, 7, 1, 14]
new makespan : 1106
=====
new seq_star : [2, 3, 18, 0, 15, 10, 9, 19, 17, 11, 13, 16, 6, 12, 4, 5, 8, 7, 14, 1]
new makespan : 1105
=====
new seq_star : [2, 3, 18, 13, 9, 15, 10, 0, 19, 6, 16, 17, 11, 12, 4, 5, 8, 7, 1, 14]
new makespan : 1103
=====
new seq_star : [2, 3, 18, 13, 9, 15, 10, 0, 19, 6, 17, 12, 4, 11, 7, 16, 8, 5, 1, 14]
new makespan : 1102
=====
new seq_star : [2, 3, 18, 13, 9, 15, 10, 12, 19, 17, 6, 11, 0, 16, 4, 5, 8, 7, 1, 14]
new makespan : 1100
=====
new seq_star : [2, 3, 18, 13, 12, 15, 10, 0, 19, 17, 4, 9, 11, 5, 6, 16, 8, 7, 14, 1]
new makespan : 1098
=====
new seq_star : [2, 3, 18, 13, 12, 15, 10, 0, 19, 17, 4, 6, 9, 11, 7, 16, 8, 5, 14, 1]
new makespan : 1097
=====
new seq_star : [2, 3, 18, 13, 12, 15, 10, 0, 19, 17, 6, 4, 9, 11, 7, 16, 8, 5, 14, 1]
new makespan : 1095
=====
new seq_star : [2, 3, 18, 13, 12, 15, 10, 0, 19, 17, 6, 11, 9, 4, 16, 5, 8, 7, 14, 1]
new makespan : 1090
=====
new seq_star : [2, 3, 18, 13, 12, 15, 10, 17, 9, 19, 11, 4, 0, 6, 16, 5, 8, 7, 14, 1]
new makespan : 1089
=====
new seq_star : [2, 3, 15, 0, 19, 17, 6, 13, 12, 18, 4, 9, 11, 8, 16, 10, 5, 7, 14, 1]
new makespan : 1088
=====
new seq_star : [2, 3, 15, 0, 19, 17, 6, 13, 12, 18, 4, 9, 11, 10, 16, 5, 8, 7, 14, 1]
new makespan : 1085
=====
new seq_star : [2, 3, 15, 0, 19, 17, 6, 13, 12, 18, 11, 4, 8, 9, 16, 10, 5, 7, 14, 1]
new makespan : 1081
=====
Execution time: 5116.9065589904785
seq_star: [2, 3, 15, 0, 19, 17, 6, 13, 12, 18, 11, 4, 8, 9, 16, 10, 5, 7, 14, 1]
MakeSpan: 1081

```

```
In [ ]: exec_instance(3)
```

```

Starting optim of the 20 jobs ...
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 5, 17, 14, 9, 1, 11, 4, 6, 0, 2]
new makespan : 1455
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 5, 17, 14, 9, 1, 11, 6, 0, 4, 2]
new makespan : 1443
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 5, 17, 6, 9, 11, 14, 1, 4, 0, 2]
new makespan : 1440
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 5, 17, 6, 9, 11, 14, 1, 0, 4, 2]
new makespan : 1438
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 5, 17, 6, 9, 0, 4, 11, 14, 1, 2]
new makespan : 1434
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 5, 17, 6, 9, 0, 4, 14, 11, 1, 2]
new makespan : 1430
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 5, 17, 6, 9, 1, 2, 11, 14, 0, 4]
new makespan : 1427
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 5, 14, 9, 1, 11, 4, 6, 0, 2, 17]
new makespan : 1425
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 5, 14, 9, 1, 11, 17, 6, 0, 4, 2]
new makespan : 1415
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 5, 14, 9, 1, 11, 6, 0, 4, 2, 17]
new makespan : 1413
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 5, 9, 1, 11, 14, 0, 4, 6, 2, 17]
new makespan : 1408
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 5, 6, 1, 2, 9, 11, 14, 0, 4, 17]
new makespan : 1407
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 14, 9, 0, 11, 17, 6, 4, 1, 2, 5]
new makespan : 1401
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 14, 9, 0, 11, 17, 6, 1, 5, 4, 2]
new makespan : 1400
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 14, 9, 0, 11, 6, 4, 1, 2, 5, 17]
new makespan : 1394
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 14, 9, 0, 4, 6, 1, 2, 5, 11, 17]
new makespan : 1387
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 14, 9, 0, 6, 4, 1, 2, 5, 11, 17]
new makespan : 1380
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 14, 9, 0, 6, 1, 11, 4, 2, 5, 17]
new makespan : 1379
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 14, 9, 1, 11, 4, 6, 0, 2, 5, 17]
new makespan : 1377
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 14, 9, 1, 11, 6, 4, 0, 2, 5, 17]
new makespan : 1372
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 16, 18, 14, 9, 1, 11, 6, 0, 4, 2, 5, 17]
new makespan : 1368
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 6, 11, 14, 16, 18, 9, 1, 0, 4, 2, 5, 17]
new makespan : 1362
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 7, 6, 16, 18, 14, 9, 11, 1, 0, 4, 2, 5, 17]
new makespan : 1360
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 9, 0, 11, 14, 1, 16, 18, 6, 4, 7, 2, 5, 17]
new makespan : 1358
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 9, 0, 14, 16, 18, 6, 1, 11, 4, 7, 2, 5, 17]
new makespan : 1351
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 9, 16, 18, 14, 6, 11, 1, 0, 4, 7, 2, 5, 17]
new makespan : 1346
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 9, 1, 11, 18, 14, 16, 6, 0, 4, 7, 2, 5, 17]
new makespan : 1339

```



```

=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 14, 16, 6, 0, 18, 9, 1, 11, 4, 7, 2, 5, 17]
new makespan : 1337
=====
new seq_star : [12, 8, 15, 13, 3, 19, 10, 14, 16, 6, 18, 9, 11, 1, 0, 4, 7, 2, 5, 17]
new makespan : 1335
=====
new seq_star : [12, 8, 15, 13, 3, 14, 16, 10, 6, 19, 18, 9, 0, 1, 11, 4, 7, 2, 5, 17]
new makespan : 1334
=====
new seq_star : [12, 8, 15, 13, 3, 14, 10, 9, 16, 18, 6, 19, 1, 0, 4, 7, 11, 2, 5, 17]
new makespan : 1333
=====
new seq_star : [12, 8, 15, 13, 3, 14, 9, 16, 18, 6, 19, 10, 1, 0, 4, 7, 11, 2, 5, 17]
new makespan : 1332
=====
new seq_star : [12, 8, 15, 13, 3, 14, 9, 16, 18, 10, 6, 19, 1, 0, 4, 7, 11, 2, 5, 17]
new makespan : 1329
=====
new seq_star : [12, 8, 15, 13, 11, 17, 14, 10, 9, 16, 18, 6, 19, 1, 0, 4, 7, 2, 5, 3]
new makespan : 1324
=====
new seq_star : [12, 8, 15, 13, 11, 14, 10, 9, 16, 18, 1, 19, 6, 4, 0, 2, 7, 3, 17, 5]
new makespan : 1323
=====
new seq_star : [12, 8, 15, 13, 11, 14, 10, 9, 16, 18, 6, 19, 1, 4, 0, 2, 7, 3, 17, 5]
new makespan : 1320
=====
new seq_star : [12, 8, 15, 13, 11, 14, 10, 9, 16, 18, 6, 19, 1, 4, 0, 2, 7, 5, 3, 17]
new makespan : 1319
=====
new seq_star : [12, 8, 15, 13, 11, 14, 10, 9, 16, 18, 6, 19, 1, 0, 4, 2, 7, 5, 3, 17]
new makespan : 1318
=====
new seq_star : [12, 8, 15, 13, 11, 14, 10, 9, 16, 18, 6, 19, 1, 0, 4, 3, 7, 5, 2, 17]
new makespan : 1315
=====
new seq_star : [12, 8, 15, 13, 17, 10, 14, 16, 6, 19, 1, 11, 18, 9, 0, 4, 7, 2, 5, 3]
new makespan : 1313
=====
new seq_star : [12, 8, 15, 13, 10, 19, 6, 11, 14, 16, 18, 9, 1, 0, 4, 3, 7, 5, 2, 17]
new makespan : 1311
=====
new seq_star : [12, 8, 15, 13, 10, 19, 6, 16, 14, 18, 9, 11, 1, 0, 4, 3, 7, 5, 2, 17]
new makespan : 1310
=====
new seq_star : [12, 8, 15, 6, 13, 19, 14, 10, 16, 18, 9, 1, 0, 4, 7, 11, 2, 5, 3, 17]
new makespan : 1309
=====
new seq_star : [12, 8, 15, 6, 16, 10, 0, 4, 11, 18, 14, 9, 19, 1, 2, 7, 5, 13, 3, 17]
new makespan : 1307
=====
new seq_star : [12, 8, 15, 6, 16, 10, 0, 4, 11, 18, 19, 14, 9, 1, 2, 7, 5, 13, 3, 17]
new makespan : 1304
=====
new seq_star : [12, 8, 15, 6, 16, 10, 0, 4, 18, 9, 11, 14, 19, 1, 2, 7, 5, 13, 3, 17]
new makespan : 1302
=====
new seq_star : [12, 8, 15, 6, 16, 10, 0, 11, 18, 14, 9, 19, 1, 4, 7, 13, 2, 5, 3, 17]
new makespan : 1301
=====
new seq_star : [12, 8, 15, 6, 16, 10, 0, 18, 9, 4, 11, 14, 19, 1, 7, 13, 2, 5, 3, 17]
new makespan : 1299
=====
new seq_star : [12, 16, 3, 8, 15, 10, 6, 18, 9, 4, 19, 1, 0, 11, 14, 7, 5, 13, 2, 17]
new makespan : 1298
=====
new seq_star : [12, 16, 8, 10, 1, 3, 15, 18, 9, 0, 19, 6, 4, 11, 14, 7, 5, 13, 2, 17]
new makespan : 1297
=====
new seq_star : [12, 16, 18, 8, 15, 0, 10, 6, 4, 11, 14, 9, 19, 1, 3, 7, 5, 13, 2, 17]
new makespan : 1296
=====
new seq_star : [12, 16, 18, 8, 15, 0, 10, 6, 4, 11, 14, 9, 19, 1, 13, 3, 7, 5, 2, 17]
new makespan : 1295
=====
new seq_star : [12, 16, 18, 8, 15, 0, 10, 6, 19, 4, 11, 14, 9, 1, 13, 3, 7, 5, 2, 17]
new makespan : 1294
=====
new seq_star : [12, 16, 18, 8, 15, 6, 0, 19, 10, 11, 14, 1, 3, 9, 4, 7, 5, 13, 2, 17]
new makespan : 1293

```

```
=====
```

```
Execution time: 6407.46325469017
```

```
seq_star: [12, 16, 18, 8, 15, 6, 0, 19, 10, 11, 14, 1, 3, 9, 4, 7, 5, 13, 2, 17]
```

```
MakeSpan: 1293
```

```
In [ ]: exec_instance(4)
```

```

Starting optim of the 20 jobs ...
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 6, 17, 18, 16, 13, 14, 15, 5, 10, 1, 4]
new makespan : 1395
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 6, 17, 18, 16, 13, 14, 15, 5, 1, 4, 10]
new makespan : 1372
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 6, 17, 18, 16, 13, 14, 15, 5, 4, 1, 10]
new makespan : 1360
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 6, 17, 18, 16, 15, 14, 5, 1, 13, 4, 10]
new makespan : 1354
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 6, 17, 18, 16, 15, 14, 5, 1, 4, 13, 10]
new makespan : 1351
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 6, 17, 18, 16, 15, 14, 5, 4, 1, 13, 10]
new makespan : 1338
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 6, 18, 16, 15, 5, 4, 1, 10, 14, 13, 17]
new makespan : 1336
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 6, 18, 16, 1, 15, 4, 5, 17, 10, 14, 13]
new makespan : 1332
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 14, 18, 5, 16, 15, 1, 10, 17, 4, 13, 6]
new makespan : 1331
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 14, 18, 5, 4, 16, 15, 1, 10, 17, 13, 6]
new makespan : 1326
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 14, 18, 16, 15, 5, 10, 1, 17, 4, 13, 6]
new makespan : 1321
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 14, 18, 16, 15, 5, 10, 1, 4, 17, 13, 6]
new makespan : 1318
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 18, 16, 15, 5, 4, 1, 17, 10, 14, 13, 6]
new makespan : 1317
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 8, 3, 18, 16, 15, 5, 4, 1, 10, 14, 13, 17, 6]
new makespan : 1312
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 18, 8, 16, 3, 1, 15, 4, 5, 17, 10, 14, 13, 6]
new makespan : 1308
=====
new seq_star : [2, 0, 7, 9, 11, 12, 19, 18, 8, 4, 3, 16, 1, 15, 5, 17, 10, 14, 13, 6]
new makespan : 1305
=====
new seq_star : [2, 0, 7, 9, 11, 12, 18, 8, 3, 5, 16, 15, 6, 4, 1, 10, 14, 13, 17, 19]
new makespan : 1304
=====
new seq_star : [2, 0, 7, 9, 11, 12, 18, 8, 3, 5, 16, 15, 1, 6, 4, 10, 14, 13, 17, 19]
new makespan : 1302
=====
new seq_star : [2, 0, 7, 9, 11, 12, 18, 8, 3, 5, 16, 4, 15, 6, 1, 10, 14, 13, 17, 19]
new makespan : 1300
=====
new seq_star : [2, 0, 7, 9, 11, 12, 18, 8, 3, 17, 16, 4, 1, 15, 5, 10, 14, 13, 6, 19]
new makespan : 1294
=====
new seq_star : [2, 0, 7, 9, 11, 12, 18, 8, 3, 6, 16, 4, 1, 15, 5, 10, 14, 13, 17, 19]
new makespan : 1293
=====
new seq_star : [2, 0, 7, 9, 11, 12, 18, 8, 3, 16, 14, 15, 4, 5, 10, 1, 13, 17, 6, 19]
new makespan : 1286
=====
new seq_star : [2, 0, 7, 9, 11, 12, 18, 8, 3, 16, 1, 15, 5, 6, 4, 10, 14, 13, 17, 19]
new makespan : 1285
=====
new seq_star : [2, 0, 7, 9, 11, 12, 18, 8, 3, 16, 1, 15, 5, 4, 10, 14, 13, 17, 6, 19]
new makespan : 1283
=====
new seq_star : [2, 0, 7, 9, 11, 12, 18, 8, 3, 16, 1, 15, 5, 4, 10, 14, 13, 6, 17, 19]
new makespan : 1281
=====
new seq_star : [2, 0, 7, 9, 11, 18, 8, 4, 16, 3, 15, 1, 12, 10, 14, 13, 5, 6, 17, 19]
new makespan : 1279
=====

```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[12], line 1
----> 1 exec_instance(4)

Cell In[8], line 7, in exec_instance(k)
      5 print("Starting optim of the 20 jobs ...")
      6 start = time.time()
----> 7 fs.optim(debug=True)
      8 print("Execution time: ", time.time() - start)
      9 print("Seq_star:", fs.seq_star)

Cell In[3], line 124, in FlowShopBB.optim(self, debug)
     121 cumu = lb_seq_cumul_tuple[2]
     122 if lb < self.upper_bound:
     123     # cumu = self.cumulate(self.jobs_list[seq[-1]], previous_cumul=None) # here previous_cumul is none
--> 124     self.bb(seq, cumu)

Cell In[3], line 99, in FlowShopBB.bb(self, job_seq, previous_cumul)
      97 cumu = lb_seq_cumul_tuple[2]
      98 if lb < self.upper_bound:
----> 99     self.bb(seq, cumu)

Cell In[3], line 99, in FlowShopBB.bb(self, job_seq, previous_cumul)
      97 cumu = lb_seq_cumul_tuple[2]
      98 if lb < self.upper_bound:
----> 99     self.bb(seq, cumu)

[... skipping similar frames: FlowShopBB.bb at line 99 (13 times)]

Cell In[3], line 99, in FlowShopBB.bb(self, job_seq, previous_cumul)
      97 cumu = lb_seq_cumul_tuple[2]
      98 if lb < self.upper_bound:
----> 99     self.bb(seq, cumu)

Cell In[3], line 89, in FlowShopBB.bb(self, job_seq, previous_cumul)
      87 augmented_partial_set.append(i)
      88 cumu = self.cumulate(self.jobs_list[i], previous_cumul)
----> 89     augmented_job_seqs.append( ( self.lower_bound(augmented_partial_set, cumu), augmented_partial_set, cumu ) )
      92 ordered_augmented_job_seqs = sorted(augmented_job_seqs)
      94 for lb_seq_cumul_tuple in ordered_augmented_job_seqs:

Cell In[3], line 45, in FlowShopBB.lower_bound(self, jobs_sequence, jobs_seq_cumul, previous_cumul)
      42 counter += 1
      44 lb_machine = cumulative_time_to_machine + remaining_machine_time + min(remaining_time)
----> 45     lb = max(lb, lb_machine)
      47 return lb

KeyboardInterrupt:

```

```
In [ ]: exec_instance(5)
```

```
Starting optim of the 20 jobs ...
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 3, 7, 13, 17, 9, 6, 8, 11, 14, 15, 18, 2]
new makespan : 1272
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 3, 7, 13, 17, 9, 6, 11, 8, 14, 15, 18, 2]
new makespan : 1270
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 3, 7, 6, 11, 13, 17, 9, 14, 15, 8, 18, 2]
new makespan : 1268
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 3, 7, 6, 11, 13, 17, 9, 15, 14, 8, 18, 2]
new makespan : 1262
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 3, 17, 13, 6, 11, 14, 8, 7, 9, 15, 18, 2]
new makespan : 1258
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 3, 6, 7, 11, 13, 17, 9, 15, 14, 8, 18, 2]
new makespan : 1252
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 6, 3, 7, 13, 17, 9, 11, 15, 14, 8, 18, 2]
new makespan : 1249
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 6, 3, 7, 13, 17, 9, 15, 14, 11, 8, 18, 2]
new makespan : 1245
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 6, 3, 7, 13, 17, 9, 15, 14, 8, 11, 18, 2]
new makespan : 1241
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 6, 3, 7, 11, 13, 17, 9, 15, 14, 8, 18, 2]
new makespan : 1236
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 6, 3, 7, 15, 17, 13, 14, 9, 11, 8, 18, 2]
new makespan : 1235
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 6, 3, 13, 17, 9, 11, 7, 15, 14, 8, 18, 2]
new makespan : 1234
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 6, 3, 13, 17, 9, 15, 14, 8, 7, 11, 18, 2]
new makespan : 1228
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 6, 7, 9, 11, 14, 15, 8, 3, 17, 13, 18, 2]
new makespan : 1226
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 6, 7, 11, 8, 13, 14, 3, 17, 9, 15, 18, 2]
new makespan : 1225
=====
new seq_star : [1, 10, 4, 0, 19, 16, 12, 5, 6, 7, 11, 8, 14, 3, 13, 17, 9, 15, 18, 2]
new makespan : 1224
=====
```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[13], line 1
----> 1 exec_instance(5)

Cell In[8], line 7, in exec_instance(k)
      5 print("Starting optim of the 20 jobs ...")
      6 start = time.time()
----> 7 fs.optim(debug=True)
      8 print("Execution time: ", time.time() - start)
      9 print("Seq_star:", fs.seq_star)

Cell In[3], line 124, in FlowShopBB.optim(self, debug)
     121 cumu = lb_seq_cumul_tuple[2]
     122 if lb < self.upper_bound:
     123     # cumu = self.cumulate(self.jobs_list[seq[-1]], previous_cumul=None) # here previous_cumul is none
--> 124     self.bb(seq, cumu)

Cell In[3], line 99, in FlowShopBB.bb(self, job_seq, previous_cumul)
      97 cumu = lb_seq_cumul_tuple[2]
      98 if lb < self.upper_bound:
----> 99     self.bb(seq, cumu)

Cell In[3], line 99, in FlowShopBB.bb(self, job_seq, previous_cumul)
      97 cumu = lb_seq_cumul_tuple[2]
      98 if lb < self.upper_bound:
----> 99     self.bb(seq, cumu)

[... skipping similar frames: FlowShopBB.bb at line 99 (12 times)]

Cell In[3], line 99, in FlowShopBB.bb(self, job_seq, previous_cumul)
      97 cumu = lb_seq_cumul_tuple[2]
      98 if lb < self.upper_bound:
----> 99     self.bb(seq, cumu)

Cell In[3], line 89, in FlowShopBB.bb(self, job_seq, previous_cumul)
      87 augmented_partial_set.append(i)
      88 cumu = self.cumulate(self.jobs_list[i], previous_cumul)
----> 89     augmented_job_seqs.append( ( self.lower_bound(augmented_partial_set, cumu), augmented_partial_set, cumu ) )
      92 ordered_augmented_job_seqs = sorted(augmented_job_seqs)
      94 for lb_seq_cumul_tuple in ordered_augmented_job_seqs:

Cell In[3], line 45, in FlowShopBB.lower_bound(self, jobs_sequence, jobs_seq_cumul, previous_cumul)
      42 counter += 1
      44 lb_machine = cumulative_time_to_machine + remaining_machine_time + min(remaining_time)
----> 45     lb = max(lb, lb_machine)
      47 return lb

KeyboardInterrupt:

```

```
In [ ]: exec_instance(6)
```

```

Starting optim of the 20 jobs ...
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 3, 19, 4, 11, 14, 17, 7, 10, 5, 2, 6]
new makespan : 1353
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 3, 19, 4, 11, 14, 2, 7, 10, 5, 6, 17]
new makespan : 1327
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 3, 19, 4, 11, 14, 2, 10, 5, 7, 17, 6]
new makespan : 1317
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 3, 19, 4, 11, 2, 14, 10, 5, 7, 17, 6]
new makespan : 1315
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 3, 19, 4, 11, 2, 7, 10, 5, 17, 6, 14]
new makespan : 1312
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 3, 19, 4, 14, 7, 10, 5, 2, 6, 17, 11]
new makespan : 1306
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 3, 19, 4, 14, 7, 5, 10, 2, 6, 17, 11]
new makespan : 1302
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 3, 19, 4, 14, 10, 5, 2, 7, 6, 17, 11]
new makespan : 1301
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 3, 19, 4, 14, 10, 5, 2, 7, 11, 17, 6]
new makespan : 1291
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 3, 4, 11, 19, 5, 2, 7, 10, 17, 6, 14]
new makespan : 1286
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 11, 19, 2, 7, 3, 10, 5, 4, 14, 6, 17]
new makespan : 1279
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 19, 4, 3, 14, 11, 10, 5, 2, 7, 6, 17]
new makespan : 1276
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 19, 4, 3, 10, 5, 2, 6, 7, 14, 11, 17]
new makespan : 1272
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 16, 8, 19, 4, 6, 3, 2, 7, 5, 10, 14, 11, 17]
new makespan : 1271
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 19, 10, 2, 3, 14, 16, 4, 5, 7, 8, 11, 6, 17]
new makespan : 1265
=====
new seq_star : [9, 1, 12, 0, 18, 13, 15, 19, 10, 2, 3, 16, 4, 5, 6, 8, 7, 14, 11, 17]
new makespan : 1263
=====
new seq_star : [9, 1, 12, 0, 18, 13, 19, 14, 3, 16, 8, 4, 7, 5, 10, 2, 6, 17, 11, 15]
new makespan : 1262
=====
new seq_star : [9, 1, 12, 0, 18, 13, 19, 14, 3, 16, 8, 4, 10, 5, 2, 7, 11, 17, 6, 15]
new makespan : 1259
=====
new seq_star : [9, 1, 12, 0, 18, 13, 19, 14, 15, 16, 8, 10, 3, 4, 11, 7, 5, 2, 6, 17]
new makespan : 1258
=====
new seq_star : [9, 1, 12, 0, 18, 13, 19, 14, 15, 16, 8, 10, 3, 4, 11, 5, 2, 7, 6, 17]
new makespan : 1253
=====
new seq_star : [9, 1, 12, 0, 18, 13, 19, 14, 15, 16, 8, 10, 3, 4, 11, 5, 7, 2, 6, 17]
new makespan : 1247
=====
new seq_star : [9, 1, 12, 0, 18, 16, 15, 19, 8, 10, 3, 4, 11, 7, 5, 2, 14, 6, 17, 13]
new makespan : 1244
=====
new seq_star : [9, 1, 12, 0, 18, 16, 15, 19, 8, 10, 3, 4, 11, 5, 7, 2, 14, 6, 17, 13]
new makespan : 1243
=====
new seq_star : [9, 1, 12, 0, 18, 16, 15, 19, 14, 2, 3, 4, 7, 10, 5, 8, 6, 11, 13, 17]
new makespan : 1242
=====
new seq_star : [9, 1, 12, 0, 18, 16, 15, 19, 14, 2, 7, 3, 4, 13, 10, 5, 8, 6, 11, 17]
new makespan : 1234
=====
Execution time: 18.09609317779541
seq_star: [9, 1, 12, 0, 18, 16, 15, 19, 14, 2, 7, 3, 4, 13, 10, 5, 8, 6, 11, 17]
MakeSpan: 1234

```

In []: `exec_instance(7)`

```

Starting optim of the 20 jobs ...
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 10, 9, 15, 17, 12, 13, 7, 19, 14, 18, 0]
new makespan : 1404
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 10, 9, 15, 17, 12, 13, 7, 19, 18, 0, 14]
new makespan : 1383
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 10, 9, 15, 17, 12, 13, 7, 18, 14, 19, 0]
new makespan : 1324
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 10, 9, 15, 17, 12, 13, 18, 7, 14, 19, 0]
new makespan : 1287
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 10, 9, 15, 17, 12, 18, 14, 13, 7, 19, 0]
new makespan : 1284
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 10, 9, 15, 17, 13, 18, 7, 19, 14, 12, 0]
new makespan : 1271
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 10, 15, 13, 18, 7, 9, 17, 19, 14, 12, 0]
new makespan : 1262
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 10, 15, 18, 7, 9, 13, 17, 19, 14, 12, 0]
new makespan : 1261
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 10, 15, 18, 7, 19, 9, 17, 14, 13, 12, 0]
new makespan : 1260
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 12, 9, 17, 15, 13, 18, 7, 19, 10, 14, 0]
new makespan : 1255
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 12, 9, 17, 15, 0, 18, 14, 13, 7, 19, 10]
new makespan : 1250
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 12, 9, 0, 15, 13, 18, 7, 19, 14, 17, 10]
new makespan : 1241
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 12, 9, 0, 15, 13, 18, 14, 17, 7, 19, 10]
new makespan : 1240
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 12, 15, 18, 7, 9, 13, 17, 0, 14, 19, 10]
new makespan : 1239
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 12, 15, 18, 7, 9, 13, 0, 19, 14, 17, 10]
new makespan : 1231
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 12, 15, 18, 7, 19, 0, 9, 13, 14, 17, 10]
new makespan : 1230
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 6, 15, 17, 18, 7, 9, 13, 0, 19, 14, 12, 10]
new makespan : 1228
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 9, 0, 15, 13, 18, 7, 14, 19, 17, 6, 12, 10]
new makespan : 1227
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 13, 7, 15, 18, 6, 17, 9, 0, 19, 14, 12, 10]
new makespan : 1223
=====
new seq_star : [11, 5, 16, 1, 4, 8, 2, 3, 15, 18, 7, 6, 13, 9, 17, 19, 0, 14, 12, 10]
new makespan : 1222
=====

```



```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 exec_instance(7)

Cell In[8], line 7, in exec_instance(k)
      5 print("Starting optim of the 20 jobs ...")
      6 start = time.time()
----> 7 fs.optim(debug=True)
      8 print("Execution time: ", time.time() - start)
      9 print("Seq_star:", fs.seq_star)

Cell In[3], line 124, in FlowShopBB.optim(self, debug)
     121 cumu = lb_seq_cumul_tuple[2]
     122 if lb < self.upper_bound:
     123     # cumu = self.cumulate(self.jobs_list[seq[-1]], previous_cumul=None) # here previous_cumul is none
--> 124     self.bb(seq, cumu)

Cell In[3], line 99, in FlowShopBB.bb(self, job_seq, previous_cumul)
      97 cumu = lb_seq_cumul_tuple[2]
      98 if lb < self.upper_bound:
----> 99     self.bb(seq, cumu)

Cell In[3], line 99, in FlowShopBB.bb(self, job_seq, previous_cumul)
      97 cumu = lb_seq_cumul_tuple[2]
      98 if lb < self.upper_bound:
----> 99     self.bb(seq, cumu)

[... skipping similar frames: FlowShopBB.bb at line 99 (8 times)]

Cell In[3], line 99, in FlowShopBB.bb(self, job_seq, previous_cumul)
      97 cumu = lb_seq_cumul_tuple[2]
      98 if lb < self.upper_bound:
----> 99     self.bb(seq, cumu)

Cell In[3], line 89, in FlowShopBB.bb(self, job_seq, previous_cumul)
      87 augmented_partial_set.append(i)
      88 cumu = self.cumulate(self.jobs_list[i], previous_cumul)
----> 89     augmented_job_seqs.append( ( self.lower_bound(augmented_partial_set, cumu), augmented_partial_set, cumu ) )
      92 ordered_augmented_job_seqs = sorted(augmented_job_seqs)
      94 for lb_seq_cumul_tuple in ordered_augmented_job_seqs:

Cell In[3], line 45, in FlowShopBB.lower_bound(self, jobs_sequence, jobs_seq_cumul, previous_cumul)
      42 counter += 1
      44 lb_machine = cumulative_time_to_machine + remaining_machine_time + min(remaining_time)
----> 45     lb = max(lb, lb_machine)
      47 return lb

KeyboardInterrupt:

```

```
In [ ]: exec_instance(8)
```

```

Starting optim of the 20 jobs ...
new seq_star : [3, 1, 7, 15, 6, 14, 11, 9, 19, 0, 10, 2, 5, 12, 17, 16, 8, 13, 4, 18]
new makespan : 1305
=====
new seq_star : [3, 1, 7, 15, 6, 14, 11, 9, 19, 0, 10, 2, 5, 12, 17, 16, 8, 18, 4, 13]
new makespan : 1302
=====
new seq_star : [3, 1, 7, 15, 6, 14, 11, 9, 19, 0, 10, 2, 5, 12, 17, 16, 13, 18, 8, 4]
new makespan : 1296
=====
new seq_star : [3, 1, 7, 15, 6, 14, 11, 9, 19, 0, 10, 2, 5, 12, 17, 16, 18, 13, 8, 4]
new makespan : 1293
=====
new seq_star : [3, 1, 7, 15, 6, 14, 11, 9, 19, 0, 10, 2, 5, 13, 12, 17, 16, 18, 8, 4]
new makespan : 1289
=====
new seq_star : [3, 1, 7, 15, 6, 14, 11, 9, 19, 0, 10, 2, 5, 17, 16, 12, 18, 13, 8, 4]
new makespan : 1286
=====
new seq_star : [3, 1, 7, 15, 6, 14, 11, 9, 19, 0, 10, 2, 5, 17, 16, 18, 12, 13, 8, 4]
new makespan : 1285
=====
new seq_star : [3, 1, 7, 15, 6, 14, 11, 9, 19, 0, 10, 17, 16, 5, 13, 12, 18, 2, 8, 4]
new makespan : 1283
=====
new seq_star : [3, 1, 7, 15, 6, 14, 9, 11, 0, 10, 2, 19, 5, 12, 17, 16, 18, 13, 8, 4]
new makespan : 1280
=====
new seq_star : [3, 1, 7, 15, 6, 14, 9, 11, 0, 10, 2, 19, 5, 13, 12, 17, 16, 18, 8, 4]
new makespan : 1279
=====
new seq_star : [3, 1, 7, 15, 6, 14, 9, 11, 0, 10, 17, 2, 5, 16, 8, 19, 18, 12, 13, 4]
new makespan : 1272
=====
new seq_star : [3, 1, 7, 15, 6, 14, 9, 11, 0, 10, 17, 2, 5, 16, 12, 18, 19, 13, 8, 4]
new makespan : 1269
=====
new seq_star : [3, 1, 7, 15, 6, 11, 9, 4, 19, 0, 10, 17, 16, 5, 2, 8, 12, 14, 18, 13]
new makespan : 1265
=====
new seq_star : [3, 1, 7, 15, 6, 11, 9, 4, 19, 0, 10, 17, 16, 5, 2, 8, 14, 13, 12, 18]
new makespan : 1263
=====
new seq_star : [3, 1, 7, 15, 6, 11, 9, 4, 19, 17, 0, 16, 5, 10, 2, 8, 14, 13, 12, 18]
new makespan : 1261
=====
new seq_star : [3, 1, 7, 15, 6, 11, 9, 19, 0, 4, 10, 17, 2, 5, 8, 14, 16, 13, 12, 18]
new makespan : 1253
=====
new seq_star : [3, 1, 7, 15, 6, 11, 9, 19, 0, 5, 10, 2, 13, 12, 14, 17, 16, 18, 8, 4]
new makespan : 1251
=====
new seq_star : [3, 1, 7, 15, 6, 11, 9, 19, 0, 5, 10, 2, 14, 13, 12, 17, 16, 18, 8, 4]
new makespan : 1247
=====
new seq_star : [3, 1, 7, 15, 6, 11, 9, 19, 0, 5, 10, 17, 2, 8, 14, 16, 12, 18, 13, 4]
new makespan : 1244
=====

```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[24], line 1
----> 1 exec_instance(8)

Cell In[12], line 7, in exec_instance(k)
      5 print("Starting optim of the 20 jobs ...")
      6 start = time.time()
----> 7 fs.optim(debug=True)
      8 print("Execution time: ", time.time() - start)
      9 print("seq_star:", fs.seq_star)

Cell In[1], line 125, in FlowShopBB.optim(self, debug)
     123 cumu = lb_seq_cumul_tuple[2]
     124 if lb < self.upper_bound:
--> 125     self.bb(seq, cumu)

Cell In[1], line 101, in FlowShopBB.bb(self, job_seq, previous_cumul)
      99 cumu = lb_seq_cumul_tuple[2]
     100 if lb < self.upper_bound:
--> 101     self.bb(seq, cumu)

Cell In[1], line 101, in FlowShopBB.bb(self, job_seq, previous_cumul)
      99 cumu = lb_seq_cumul_tuple[2]
     100 if lb < self.upper_bound:
--> 101     self.bb(seq, cumu)

[... skipping similar frames: FlowShopBB.bb at line 101 (13 times)]

Cell In[1], line 101, in FlowShopBB.bb(self, job_seq, previous_cumul)
      99 cumu = lb_seq_cumul_tuple[2]
     100 if lb < self.upper_bound:
--> 101     self.bb(seq, cumu)

Cell In[1], line 91, in FlowShopBB.bb(self, job_seq, previous_cumul)
      89 augmented_partial_set.append(i)
      90 cumu = self.cumulate(self.jobs_list[i], previous_cumul)
----> 91     augmented_job_seqs.append( ( self.lower_bound(augmented_partial_set, cumu), augmented_partial_set, cumu ) )
      94 ordered_augmented_job_seqs = sorted(augmented_job_seqs)
      96 for lb_seq_cumul_tuple in ordered_augmented_job_seqs:

Cell In[1], line 41, in FlowShopBB.lower_bound(self, jobs_sequence, jobs_seq_cumule, previous_cumul)
      39 remaining_time[counter] = 0
      40 for j in range(machine_index + 1, nb_machines):
----> 41     remaining_time[counter] += self.jobs_list[i][j]
      42     counter += 1
      44 lb_machine = cumulative_time_to_machine + remaining_machine_time + min(remaining_time)

KeyboardInterrupt:

```

```
In [ ]: exec_instance(9)
```

```

Starting optim of the 20 jobs ...
new seq_star : [6, 10, 4, 12, 11, 15, 5, 0, 1, 7, 2, 3, 9, 18, 14, 19, 16, 17, 8, 13]
new makespan : 1206
=====
new seq_star : [6, 10, 4, 12, 11, 15, 5, 0, 1, 7, 2, 3, 9, 18, 14, 13, 16, 17, 8, 19]
new makespan : 1167
=====
new seq_star : [6, 10, 4, 12, 11, 15, 5, 0, 1, 7, 2, 3, 9, 19, 18, 8, 17, 13, 16, 14]
new makespan : 1166
=====
new seq_star : [6, 10, 4, 12, 11, 15, 5, 0, 1, 7, 2, 3, 9, 19, 18, 17, 8, 13, 16, 14]
new makespan : 1159
=====
new seq_star : [6, 10, 4, 12, 11, 15, 5, 0, 1, 7, 2, 3, 9, 19, 18, 17, 13, 16, 8, 14]
new makespan : 1158
=====
new seq_star : [6, 10, 4, 12, 11, 15, 5, 0, 1, 7, 2, 16, 9, 18, 17, 13, 14, 3, 19, 8]
new makespan : 1151
=====
new seq_star : [6, 10, 4, 12, 11, 15, 5, 0, 1, 7, 2, 18, 17, 16, 9, 13, 19, 3, 14, 8]
new makespan : 1143
=====
new seq_star : [6, 10, 4, 12, 11, 15, 5, 0, 1, 7, 2, 18, 17, 9, 13, 19, 16, 3, 14, 8]
new makespan : 1127
=====

```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[25], line 1
----> 1 exec_instance(9)

Cell In[12], line 7, in exec_instance(k)
      5 print("Starting optim of the 20 jobs ...")
      6 start = time.time()
----> 7 fs.optim(debug=True)
      8 print("Execution time: ", time.time() - start)
      9 print("Seq_star:", fs.seq_star)

Cell In[1], line 125, in FlowShopBB.optim(self, debug)
     123 cumu = lb_seq_cumul_tuple[2]
     124 if lb < self.upper_bound:
--> 125     self.bb(seq, cumu)

Cell In[1], line 101, in FlowShopBB.bb(self, job_seq, previous_cumul)
      99 cumu = lb_seq_cumul_tuple[2]
     100 if lb < self.upper_bound:
--> 101     self.bb(seq, cumu)

Cell In[1], line 101, in FlowShopBB.bb(self, job_seq, previous_cumul)
      99 cumu = lb_seq_cumul_tuple[2]
     100 if lb < self.upper_bound:
--> 101     self.bb(seq, cumu)

[... skipping similar frames: FlowShopBB.bb at line 101 (9 times)]

Cell In[1], line 101, in FlowShopBB.bb(self, job_seq, previous_cumul)
      99 cumu = lb_seq_cumul_tuple[2]
     100 if lb < self.upper_bound:
--> 101     self.bb(seq, cumu)

Cell In[1], line 88, in FlowShopBB.bb(self, job_seq, previous_cumul)
      86 for i in range(len(self.jobs_list)):
      87     if i not in job_seq:
----> 88         augmented_partial_set:list = (copy.deepcopy(job_seq))
      89         augmented_partial_set.append(i)
      90         cumu = self.cumulate(self.jobs_list[i], previous_cumul)

File d:\ycondabase\Lib\copy.py:146, in deepcopy(x, memo, _nil)
     144 copier = _deepcopy_dispatch.get(cls)
     145 if copier is not None:
--> 146     y = copier(x, memo)
     147 else:
     148     if issubclass(cls, type):

File d:\ycondabase\Lib\copy.py:201, in _deepcopy_list(x, memo, deepcopy)
     198 d[weakref.ref] = _deepcopy_atomic
     199 d[property] = _deepcopy_atomic
--> 201 def _deepcopy_list(x, memo, deepcopy=deepcopy):
     202     y = []
     203     memo[id(x)] = y

```

KeyboardInterrupt: